GrowMate-BT: A Safe-by-Construction Framework for Voice-Controlled Assistive Gardening Robots that Bounds Language-Model Authority with Deterministic Behaviour Trees

RISHABH JAIN, Maynooth University, Ireland
MAJID SOROURI, Maynooth University, Ireland
GERARD LACEY, Maynooth University, Ireland
LUCIA CARRAGHER, Dundalk Institute of Technology, Ireland

Gardening supports the physical and mental well-being of older adults, yet age-related change and disability can make everyday tasks such as watering and inspecting plants difficult. Assistive robots can relieve this burden, but their screen- and menu-based interfaces often exclude the users who would benefit most. Voice is a more accessible channel, yet a misunderstood spoken request to a *physical* robot can cause real motion, device activation, or tool use, so the cost of ambiguity is physical rather than merely conversational. We present GrowMate-BT, a safe-by-construction framework for voice control of assistive robots. Where many systems let a language model generate robot plans, code, or commands, GrowMate-BT instead confines a small, locally deployed language model to flat intent classification over a fixed action vocabulary; a deterministic behaviour tree (BT) then performs all grounding, sequencing, safety checking, and terminal confirmation, so the model cannot author control flow, issue robot commands, or remove safety checks. Unlike a fixed voice-command grammar, the input stays open and paraphrased; unlike generative planners, the model gains no authority to sequence or actuate. To our knowledge, prior evaluated voice-controlled robot systems have not paired open speech with this bounded authority. We demonstrate the framework through a case study on a FarmBot Genesis XL gardening robot. In simulation, a 2,500-case command corpus attains a desired-behaviour success rate of 93.6% under a strict criterion that counts any whole-garden over-action as a failure (94.2% under a looser required-substring criterion), with an unsafe-state count of 0, measured by parsing every published command against the workspace bounds; under 120 forced misclassifications injected below the classifier, no error produced an unsafe motion or an unsupported success claim. The safety property is thus established by construction and in simulation; hardware evaluation on the physical robot (55 cases) is a smaller transfer check reported as a separate regime rather than a physical-safety validation at scale. These results come from a single run of one model configuration, so run-to-run variance under the non-deterministic classifier is not yet quantified. A formative demonstration with older-adult attendees informs the accessibility discussion rather than serving as a controlled usability study. The design keeps a modern model's language flexibility while robot structure, safety, and completion claims stay deterministic, a pattern we expect to carry to other assistive robots that expose a defined set of actions.

CCS Concepts: • **Human-centered computing** → **Human computer interaction (HCI)**; **Accessibility**; • **Computing methodologies** → *Artificial intelligence*; • **Computer systems organization** → *Robotic control*.

Additional Key Words and Phrases: assistive robotics, human–robot interaction, behaviour trees, on-device large language models, voice user interfaces, accessibility, safe robot control, agricultural robotics

Authors' Contact Information: Rishabh Jain, Maynooth University, Department of Electronic Engineering, Maynooth, Ireland, rishabh12j@gmail.com; Majid Sorouri, Maynooth University, Department of Electronic Engineering, Maynooth, Ireland, Majid.Sorouri@mu.ie; Gerard Lacey, Maynooth University, Department of Electronic Engineering, Maynooth, Ireland, Gerry.Lacey@mu.ie; Lucia Carragher, Dundalk Institute of Technology, Dundalk, Ireland, lucia.carragher@dkit.ie.

2027. Manuscript submitted to ACM

1 Introduction

Gardening promotes physical health, well-being, and continued engagement with outdoor activity [21]. However, age-related changes, disability, or reduced mobility can make repetitive tasks such as watering, inspecting plants, and moving tools difficult. Robotic systems can reduce some of this physical burden, but their interfaces may introduce a different barrier. Screen-centric applications, coordinate maps, and specialised command vocabularies assume that a user can comfortably read a display, navigate menus, and translate an everyday goal into robot-specific instructions. Those assumptions may not hold for users with reduced vision, limited digital literacy, or motor impairment. The people who could benefit from assistance may therefore be among those least well served by the interface used to request it.

Speech offers a more direct way to express goals, but voice access alone does not make a physical robot safe or accessible. Studies of voice assistants have documented recognition failures, repeated repair work, and changes in user trust after errors, particularly during sustained use by older adults [3, 14, 15]. For a conversational assistant, an incorrect response is frustrating; for a robot, a misunderstood request may also cause unexpected motion, activate a device, or select the wrong tool. This difference raises the cost of ambiguity. A user must be able to understand what the system intends to do, interrupt it without relying on the same language model that may have made the error, and trust that the robot reports success only after the requested action has actually been completed. These risks fall hardest on the users the system is meant to help: reduced mobility, balance, or reaction time can make it harder to move clear of unexpected motion, so a safety guarantee that holds even when language is misread matters most for older adults.

This work investigates how natural-language flexibility can be introduced without compromising robot safety, by letting deterministic software, rather than the language model, retain control over robot behaviour. The resulting requirements are architectural. The system should remain bounded when language interpretation is wrong; its proposed action and control structure should be inspectable before execution; safety conditions should be enforced independently of the model; and completion should be reported only after confirmation from the robot. Local processing is also desirable because spoken requests may reveal health, routine, or household information, and an assistive interface should not require cloud connectivity for its basic control path.

We present GrowMate-BT, a constraint-based architecture that separates language understanding from robot authority. Speech is transcribed and a small locally deployed language model maps the utterance to a flat, typed intent over a fixed action vocabulary. The model does not generate behaviour trees, plans, program code, command strings, or control flow. Deterministic software grounds the intent against known entities, selects a behaviour-tree template from a fixed library, and prefixes robot-touching actions, those that move the gantry or actuate a device, with the required availability, tool, workspace, and target checks. The resulting tree is inspectable and is executed through a verification gate that withholds a success response until the terminal outcome is known. Emergency-stop terms are handled before the language model is called. In short, *the LLM proposes; the behaviour tree disposes.*

We demonstrate GrowMate-BT through a case study on a FarmBot Genesis XL [7], a Cartesian gardening robot controlled through an existing ROS2 stack. The prototype adds a speech and behaviour-tree layer on top of that stack and issues commands through its established command interface, rather than replacing the underlying robot controller. FarmBot is the testbed, while the broader contribution is the division of authority: language models interpret a user's request, and deterministic, robot-specific components retain responsibility for grounding, sequencing, safety, and verification.

The contributions of this work are:

- **The GrowMate-BT architecture**, which confines a language model to flat intent classification and deterministically constructs an inspectable behaviour tree from the classified intent;
- **Safety-prefixed construction and verified feedback**, in which robot-touching actions receive explicit guards in code and success is not communicated before the robot's terminal outcome;
- **The GrowMate-BT implementation**, an additive voice-control layer for the AURA/FarmBot platform with a browser interface, intent service, behaviour-tree executor, and ROS2 bridge;
- **A safety-focused evaluation method** combining scripted command corpora with intent-injection stress testing to distinguish language errors, guard interventions, desired behaviour, and unsafe states; and
- **Formative accessibility input** from a community demonstration with older-adult attendees, used to identify practical concerns including confirmation, emergency stopping, speech robustness, affordability, and maintenance.

The claims in this paper are deliberately bounded. Safe by construction refers to the allocation of control authority and the deterministic enforcement of specified guards; it is not a guarantee against perception, mechanical, firmware, or deployment faults. The reported numbers are a single run of one model configuration, so run-to-run variance under the non-deterministic classifier is not yet measured. The technical evaluation, hardware checks, and community feedback are reported as separate evidence streams. Section 2 positions GrowMate-BT against language-driven robot safety, LLM–behaviour-tree systems, assistive voice interfaces, small-model edge deployment, and agricultural HRI. The remainder of the paper presents the proposed methodology, its implementation and evaluation, then a discussion of its broader implications, limitations, and future directions.

2 Related Work

2.1 Safety gating for language-driven robots

Safety mechanisms for language-driven robots can be placed in the model, around the planner, or in the execution system. SafeAgentBench shows that open-ended embodied planners rarely reject hazardous requests: its most safety-conscious baseline rejects only 10% of detailed hazardous tasks, and swapping the underlying language model does not notably help [25]. This motivates architectural enforcement rather than relying on a model to recognise danger from its prompt alone. KnowNo instead uses conformal prediction to align planner uncertainty with a user-selected risk level: the robot executes when its prediction set is sufficiently narrow and otherwise asks a person for help [19]. This gives calibrated uncertainty handling but does not define deterministic guards for each physical action.

SafeGate is the closest architectural counterpart to GrowMate-BT. It combines hazard analysis with a deterministic pre-execution decision gate, task safety contracts, static verification, and runtime monitoring [17]. Its safety gate nevertheless consumes an LLM-produced hazard representation before authorising, deferring, or rejecting a task. GrowMate-BT moves the trust boundary earlier: the LLM supplies only a domain intent, while deterministic grounding and safety checks are embedded inside every compiled robot-action subtree. Consequently, safety does not depend on model confidence or on the model correctly describing a hazard. Uncertainty may reduce task success or cause a refusal, but it cannot remove the checks that mediate actuation.

2.2 LLM × behaviour-tree control

Behaviour trees are increasingly used as a structured output language for LLM-based robot planning. LLM-BRAIn and BTGenBot train lightweight language models to emit complete trees from natural-language task descriptions [9, 13].

This approach offers open-ended composition, but it also makes syntax, node selection, ordering, and precondition logic learned outputs. Validation and repair therefore become necessary after generation. BTGenBot-2 strengthens this approach with an executable benchmark, dedicated validators, and a regeneration loop [10]. LLM-as-BT-Planner similarly studies several tree-generation strategies for manipulation, separating syntactic executability from goal completion [1].

The generation approach has also reached assistive and social-robot settings. Merino-Fidalgo et al. use an LLM to generate and adapt a social robot's tree at runtime, with clarification and failure interpretation supporting ambiguous or failed tasks [16]. This buys flexibility, including the ability to compose previously unspecified behaviour, but leaves the model involved in control-structure creation and revision. GrowMate-BT makes the opposite trade. A model selects one action from a fixed set of supported actions; deterministic builders then compile the only permitted tree structures from a reviewed node library. GrowMate-BT does not show that LLM-generated trees are impossible. Rather, it removes tree syntax and model-authored control flow from the failure surface, at the cost of requiring each new capability to be added explicitly in code.

2.3 Assistive voice interfaces and older-adult harms

VoicePilot demonstrates the accessibility potential of an LLM-based speech interface for a physically assistive feeding robot and, importantly, evaluates the interface with older adults [18]. Its model can produce sequences of robot functions, with parameter validation and platform-specific limits applied around the generated program. GrowMate-BT shares the goal of making robot control easier to express, but places the generative component behind a narrower authority boundary: it classifies an intent and cannot compose the robot program.

Longitudinal studies of everyday voice assistants explain why this boundary matters. In-home research with older adults documents activation, transcription, intent-recognition, and response errors, together with correction attempts that can create further breakdowns [14]. Longer-term work also characterises conversational repair as effort transferred to users and caregivers, especially when systems fail to explain or recover from repeated misunderstandings [15]. More general voice-assistant research links failure source to trust and continued task use [3]. These studies do not evaluate physical robots, and their observed error rates are not comparable to GrowMate-BT's scripted evaluation. They are used here as design-requirement evidence: a robot voice interface should refuse cleanly rather than guess, expose what it intends to do, offer an independent stop path, and avoid claiming completion when execution has not been confirmed.

2.4 Edge / small-model deployment

Local inference can reduce cloud dependence and limit the disclosure of spoken requests, but resource-constrained deployment changes which language-model task is practical. An edge mobile-robot study illustrates the difficulty of using a quantised local model to emit direct robot command strings reliably [20]. Studies of small-model function calling likewise identify schema adherence and parameter correctness as distinct failure modes, alongside latency and memory constraints [12]. In contrast, narrow classification work in assistive leader–follower interaction suggests that small models can be useful when the output space is deliberately bounded [2].

GrowMate-BT follows this narrower regime. The current GrowMate-BT implementation uses a locally hosted Gemma classifier, while fully browser-resident inference on a phone remains future work. In either deployment, the architectural point is the same: the model returns a small flat schema, and the robot-specific structure is constructed elsewhere. The edge literature therefore motivates both the fixed classification role and a future model-size study that reports classification accuracy, schema adherence, latency, and memory separately from robot execution time.

2.5 Voice HRI in agriculture: a limited literature

Speech in agricultural HRI is limited, but it is not literally absent. Agricultural prototypes have used speech recognition to trigger fixed picking or vehicle actions [24], while AgriVLN studies natural-language navigation in agricultural scenes using a vision–language model [26]. Related work on agricultural robot fleets has examined audio notifications for remote human supervision rather than speech as an actuation interface [11]. These systems establish that language and audio can support agricultural interaction, but they address different parts of the control problem.

The gap suggested by our July 2026 literature review is therefore narrower than an absence of voice control. To our knowledge, no prior evaluated system combines open natural-language intent interpretation, deterministic safety-prefixed execution, verified completion feedback, and an explicitly assistive agricultural use case. GrowMate-BT occupies this intersection. The novelty claim is not that it is the first machine to move or spray after a spoken keyword; it is that GrowMate-BT gives flexible language access to an agricultural robot while keeping sequencing, safety, and completion authority outside the language model.

3 Background and Preliminaries

A behaviour tree (BT) is a hierarchical control representation whose nodes are evaluated repeatedly from a root node [4]. Leaf nodes represent conditions or actions, while composite nodes determine how their children are visited. Each tick returns one of three principal states: Success when a node has achieved its purpose, Failure when it cannot or should not proceed, and Running while an operation remains in progress. A Sequence composite visits its children from left to right, stops on the first Failure or Running result, and succeeds only when every child succeeds. A Selector instead tries alternatives until one succeeds. This short-circuit behaviour makes a Sequence useful for safety gating: when a condition before an action fails, the action and all later children are never ticked. Conditions therefore mediate whether execution is permitted rather than merely reporting a warning after motion has begun.

Intent classification and plan generation give a language model substantially different authority. A classifier maps an utterance to a label from a fixed vocabulary and a small set of fields, such as a target or action parameter. A planner or code generator additionally decides ordering, branching, preconditions, and executable structure. GrowMate-BT permits the former and excludes the latter: invalid labels or malformed records can be rejected by the schema, while all robot control flow is supplied by deterministic software. Classification can still be wrong, so this restriction does not guarantee task correctness. It limits the consequences of an error by ensuring that a misclassified intent must still pass the same grounding and safety logic as a correctly classified one.

This narrower role is also relevant to local deployment. Small-model studies show that structured function calling introduces separate failures in label selection, parameter generation, and output-format adherence [12]. Keeping the learned output flat reduces the structure the model must produce and leaves recursive task composition to code. In the current system, Gemma is hosted locally through Ollama; executing the classifier directly in a phone browser is a future deployment target rather than a result claimed in this paper.

The physical testbed is a FarmBot Genesis XL, a Cartesian gantry robot with an interchangeable tool head for crop-care operations [7]. It is controlled through the existing AURA ROS2 stack, the upstream open-source software that drives this robot family. AURA exposes a command path on `keyboard_topic` and translates accepted command strings into the lower-level operations sent to the Farmduino controller. GrowMate-BT uses this single established interface: it does not replace AURA's motion or firmware layers. This separation provides the platform context for the GrowMate-BT design in Section 4 and the GrowMate-BT implementation in Section 5.

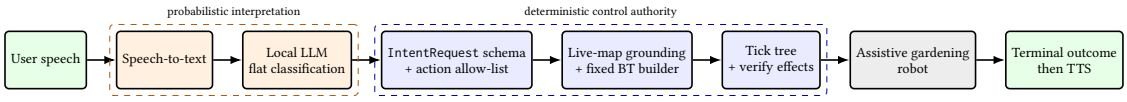


Fig. 1. GrowMate-BT enforces an authority boundary between learned interpretation and deterministic control: the learned components convert speech to a bounded intent record, and only validated, grounded, deterministic behaviour trees can issue commands. Completion language is produced after execution evidence returns from the robot path.

4 System Design and Architecture: GrowMate-BT

GrowMate-BT separates language interpretation from robot authority, as shown in Fig. 1. Speech recognition and the language model form a probabilistic front end, but neither component can publish a robot command. The language model performs *flat intent classification only*: for each clause it selects an `action` from a fixed enumeration and supplies bounded fields such as `target`, `params`, `question`, and a candidate `response`. These records are carried in an `IntentRequest` together with the original utterance, an emergency flag, and request metadata. Schema validation rejects unknown actions and malformed fields at the interface boundary. A compound utterance may produce several independent intent records in spoken order, but deterministic code, rather than the model, wraps their subtrees in a Sequence. The model therefore emits no branches, loops, retries, executable code, or safety conditions.

A fixed enumeration limits which actions the model can request, not whether it requests the right one. A misclassification can still land on a valid but wrong intent, such as watering the wrong species or moving to the wrong bed. GrowMate-BT does not prevent these; it keeps them *bounded*. Every action, right or wrong, is drawn from the same guarded vocabulary and passes the availability, tool, workspace, and target checks before any command is published, so a wrong-but-valid intent stays within the workspace and its deterministic guards rather than entering an unsafe state. Staying within bounds is not the same as being harmless: acting on the wrong target still wastes resources, and for a consequential tool such as the weeder it can damage the wrong plant. We therefore call this outcome *wrong-but-bounded* rather than safe, and Section 7 reports it as its own category. Because success follows terminal confirmation, the user hears what the robot did and can stop it or ask again.

Grounding and deterministic compilation. After validation, targets are resolved against the shared live garden map. A species or alias such as "tomatoes" becomes an ordered set of current plant coordinates; an absent target produces a refusal tree rather than a default coordinate or a guessed substitute. The selected action is then dispatched to a corresponding deterministic `_tree_*` builder. Each builder draws from a fixed, hand-authored node library and supplies the task structure that the classifier is forbidden to generate. For example, one `water` intent expands into tool preparation and a repeated per-plant sequence, as illustrated in Fig. 2. Plant traversal order, pump duration, logging, and the final response are all inserted by code. This also means that extending the action vocabulary requires an explicit schema value and a reviewed builder; prompt text alone cannot create a new robot capability.

Commands differ in how much they appear to ask for: a light toggles, a query wants a reading, a chain implies several steps. None of this changes the model's output, which stays a single action with a few bounded fields. Duration, coordinates, traversal order, and the sequencing of a chain are supplied by the builder from configuration and the live map, so a more elaborate request grows the inspectable builder logic rather than the model's authority.

A structural guard-coverage contract test builds every action's tree across all greenhouse configurations and, for each coordinate move it finds, checks its preceding siblings; this confirms that an explicit `CheckBounds` leaf precedes every live-map coordinate move, including the shortest single-plant case, as required by the design contract and shown

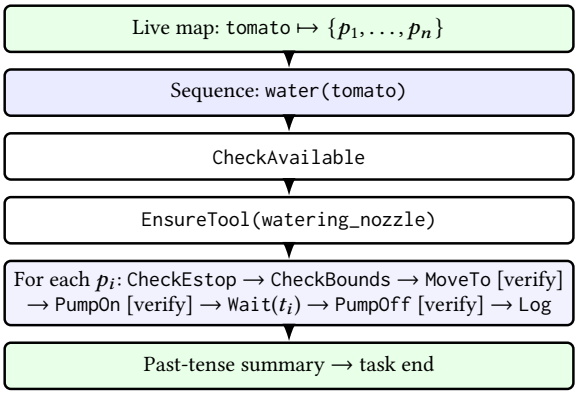


Fig. 2. Simplified deterministic tree required for "water the tomatoes." A failed guard short-circuits the Sequence; a completion statement is reached only after all verified per-plant operations succeed.

in Fig. 2. Two further checks support this: a wire-grammar contract test extracts the FarmBot controller's accepted command grammar from its source and confirms that every string the builders can publish is accepted, and a runtime pass feeds corrupted, out-of-bounds, and non-finite coordinates through the map-driven builders and confirms that the guard fails closed and no out-of-bounds motion is published, so the invariant is enforced against bad input, not merely present on valid input.

Safety and verified outcomes. Robot-touching builders prepend the guards applicable to their action. These include availability and emergency-state checks, target-existence checks, workspace bounds checks for coordinate-bearing motion, and tool readiness for operations that require a head. Sequence short-circuiting ensures that a failed guard prevents all later action nodes from being ticked. Emergency phrases are matched before classification and sent through a direct server path that latches the stop and pre-empts ordinary work; stopping the robot therefore does not depend on the model selecting the correct label.

Safety continues after command publication through a tick-and-verify gate. A motion or actuator node remains RUNNING while the robot is busy and returns FAILURE on a firmware error, timeout, or emergency stop. Completion evidence includes a firmware busy-to-idle transition, reported position for motion, and pin reads for sensor or tool operations where applicable. Services for which downstream completion is not observable are described only as requested, not completed. Consequently, the model's candidate `response` is never treated as proof of success: a past-tense confirmation and a care-event log are reached only after the corresponding tree path succeeds. On failure, the server reports the failed operation without repeating an unearned completion claim. We name this policy *honest-or-blank* (our term): the system asserts a completion only when terminal evidence supports it, and otherwise stays silent on the outcome rather than guessing.

Figure 3 makes the two enforcement points and their success and failure paths explicit.

Inspectability, scope, and alternatives. Because a task is an ordinary behaviour-tree data structure assembled from a named node library, its guards, order, and terminal responses can be inspected or rendered before execution. This is valuable for an assistive interface: a developer, caregiver, or auditor can identify the condition that blocked a request without interpreting latent model state. The claim is bounded rather than absolute: it constrains language-model authority and the hazards its guards represent, not faults outside that model.

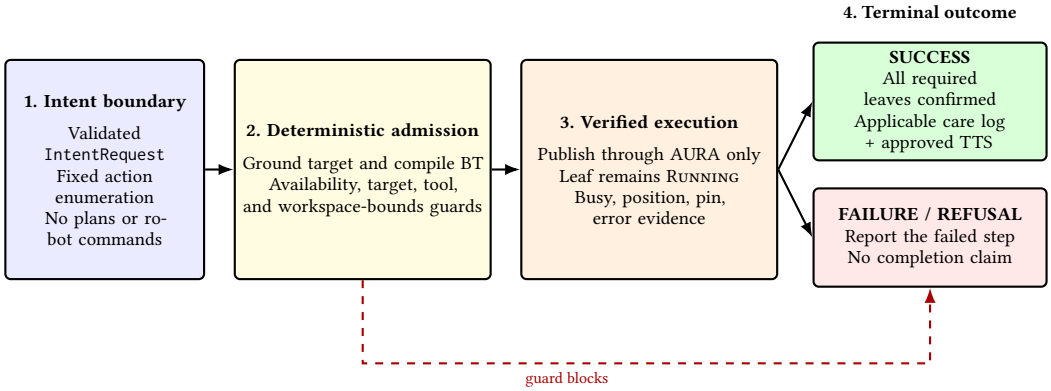


Fig. 3. Verified-execution lifecycle. Admission and verification are separate deterministic gates. A failed guard prevents command publication; after publication, an error, timeout, emergency stop, or missing terminal evidence prevents success. Only the success path may write an applicable care-event record and issue a past-tense confirmation.

We considered three broader alternatives. Allowing an LLM to generate a tree or program would return sequencing and safety structure to probabilistic output. An end-to-end learned control policy would make the command boundary and failure reason harder to inspect. A keyword grammar would preserve determinism but lose much of the paraphrase tolerance and conversational accessibility motivating the interface. GrowMate-BT instead localises platform-specific work in the action enumeration, grounding adapter, builders, command publisher, and verification adapter. The architectural recipe (bounded classification followed by deterministic, verified execution) can therefore be reused even when those robot-specific components change.

5 Implementation: GrowMate-BT on FarmBot

The case study is the FarmBot Genesis XL, an open-source Cartesian gardening robot. A tool head travels three axes over a raised bed on an aluminium-extrusion gantry under a Farmduino firmware controller, exchanging interchangeable heads (a watering nozzle, a soil-moisture probe, a seeder, and a weeder) from a fixed tool bay. Its operations are naturally enumerable and its reachable workspace is bounded, which is what lets the safe-by-construction argument be concrete here: every motion resolves to a coordinate the guards can check in advance, and every tool action carries an observable completion signal. GrowMate-BT runs on two such robots, each in a greenhouse with its own live plant map, for the motivating scenario of a user who can say what a plant needs but cannot operate the robot's native keyboard or map interface.

The GrowMate-BT reference implementation divides the pipeline between a local client host and a Raspberry Pi attached to each FarmBot. The client is a FastAPI application that serves the browser interface and accepts microphone audio, typed requests, and manual controls. Speech recognition and synthesis use pluggable backends,[1] an implementation choice rather than part of the GrowMate-BT safety argument.

After transcription, emergency phrases and a small set of exact commands are handled by deterministic pattern routes. Selected watering requests can also be held for explicit confirmation. Remaining natural-language requests

[1]Speech-to-text: Faster-Whisper (default, English small model, https://github.com/SYSTRAN/faster-whisper), Vosk (https://alphacephei.com/vosk), or Moonshine (https://github.com/usefulsensors/moonshine). Text-to-speech: Kokoro (https://github.com/hexgrad/kokoro), Piper (https://github.com/rhasspy/piper), or browser speech.

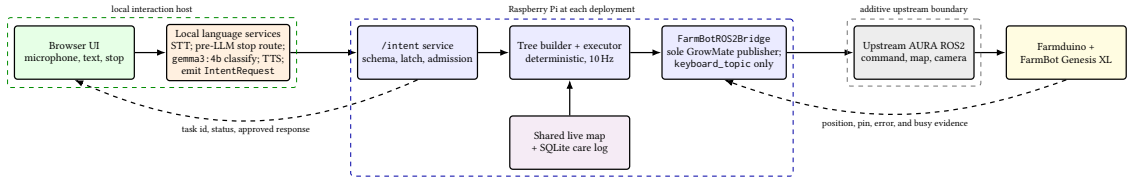


Fig. 4. Concrete GrowMate deployment and integration boundary. Language services run on a local interaction host; validated requests are executed on the Raspberry Pi associated with a deployment. GrowMate reaches the robot only through AURA's existing `keyboard_topic`, while firmware and busy-state evidence returns through the bridge before task status and approved speech are exposed to the user.

enter the intent classifier, which calls `gemma3:4b` through a locally hosted Ollama runtime. The classifier uses JSON mode, temperature 0.1, a prompt containing the fixed action vocabulary and live plant names, and a sanitizer that rejects unknown labels before constructing an `IntentRequest`. Thus, the present prototype keeps speech, classification, and synthesis on a local edge host, but does not yet run Gemma directly inside the phone browser. Browser-resident inference is retained as the deployment target discussed in Section 9.

The client sends validated requests to the Pi-side `/intent` service, which checks the schema version, refuses ordinary work while an emergency latch is set, and admits one tree at a time. It dispatches each intent to the deterministic builder of Section 4, ticks the resulting `py_trees` structure, and returns the tree outcome, leaf-level node results, commands issued, duration, and approved TTS text, with a stop control available throughout. Verified care events are written to a local event log only after the relevant action path completes.

The current schema exposes 20 labels. Motion comprises `move` and `go_home`, with browser jogging represented as a coordinate-parameterized move. Crop care comprises species watering, whole-garden watering, sensor-guided `water_smart`, raw sensor readings, and moisture queries. The remaining robot capabilities cover lights, photographs and panoramas, weed scanning and removal, bed scanning and plant-map construction, and explicit tool mounting or stowing. `emergency_stop` and `general_question` complete the interface. Unsupported requests cannot invent another label. A contract test checks that the schema enumeration, client allow-list, classifier prompt, and builder dispatch remain aligned whenever this vocabulary changes.

GrowMate-BT is integrated additively with the AURA FarmBot ROS2 stack. The `FarmBotROS2Bridge` is the only GrowMate-BT component that publishes robot commands, and it publishes exclusively to AURA's existing `keyboard_topic`. It subscribes to `busy_state` and `uart_receive` for completion, position, firmware-error, and pin-read evidence. AURA remains responsible for command handling, low-level sequencing, the Farmduino connection, mapping, and camera services; GrowMate-BT does not reimplement or replace those layers. The prototype builds on a fork of the AURA stack; integration required a small number of minimal, individually documented changes to the forked packages: serial-decode robustness, the tool-command wire grammar, the shared writable map-state directory noted above, a tool release-direction correction, and camera capture and calibration robustness. None of these alters AURA's command interface, low-level sequencing, or the GrowMate-BT control boundary, and each is consistent with the upstream project's current maintenance.

Figure 4 locates the implemented components and makes the additive boundary with the upstream robot stack explicit.

Each greenhouse selects its own workspace bounds, tool-bay geometry, soil thresholds, and scheduler settings through a deployment configuration. Mutable plant state is deliberately kept outside both source and install trees.

On each Pi, `FARMBOT_STATE_DIR` (default `~/.farmbot`) provides one writable `active_map.yaml` and watering guide shared by GrowMate-BT and AURA, preventing two components from acting on different copies of the garden. The primary deployment currently has a 56-plant map; a second deployment provides a 35-plant map. These are map populations, not evaluation sample sizes.

The work claimed comprises the GrowMate-BT constraint, wire schema and action vocabulary, classifier prompt and sanitization, browser-to-Pi wiring, Pi intent service, builders and nodes, verified executor integration, event log, and terminal-outcome interface. AURA, FarmBot hardware and firmware, Gemma and Ollama, the STT/TTS engines, ROS2, `py_trees`, and the web frameworks remain upstream or third-party contributions. All listed action paths are implemented and available to simulation, while hardware maturity is uneven: watering, tool changes, soil sensing, and the terminal-outcome path have been exercised on the physical FarmBot robot, and the hardware corpus is reported as a separate regime in Section 7. Physical camera, detection, and weed-removal accuracy fall outside the reported measurements and are noted as limitations. Section 7 therefore separates implementation coverage from measured evidence.

6 Evaluation Methodology

Evaluation objective. This is an architectural feasibility and safety evaluation, not a controlled usability study, a speech recognition benchmark, or a claim about agronomic effectiveness. It asks two bounded questions. First, does the complete text-to-intent-to-tree pipeline produce the behaviour specified by a case oracle? Second, when classification is wrong, do deterministic guards prevent unsafe execution and unsupported completion claims?

Evaluation regimes. The primary corpus run begins with text at the same intent classifier used by the application, then passes the resulting `IntentRequest` through the Pi server, grounding layer, tree builder, executor, and command bridge. It uses the fixed `gemma3:4b` model and the primary deployment's configuration and live plant map. The Pi server runs without ROS2 hardware; its simulation bridge reproduces the command grammar, busy cycles, position reports, pin responses, and tool-state sequencing needed to exercise the verification logic. This regime evaluates classification, grounding, tree construction, guard behaviour, command traces, and terminal feedback, but not microphone audio, STT word-error rate, physical motion, perception accuracy, or real tool seating. Any hardware run is reported separately and never pooled with simulation.

Corpus construction. The current validation corpus contains 2,500 cases across 12 categories: direct, indirect, query, general, emergency, multi-action, safety, refusal, hard, negation, out-of-scope, and tool commands. It is generated deterministically with seed 42 and grounded in the species and command vocabulary of the primary deployment. The first wave preserves 43 hand-authored regression seeds and expands them with controlled paraphrase, politeness, filler, disfluency, negation, and recognition-error transformations. A second wave contributes 380 synthetic cases informed by anonymized speech phenomena noted during formative community demonstrations, including plant-name hesitation, restarts, self-correction, indirect requests, and plausible recognition errors. No name, personal circumstance, raw audio, participant transcript, or direct quotation is included. These cases are authored test variations, not participant data.

The corpus was developed alongside the implementation and used to expose and repair classifier, grounding, and harness weaknesses; it is therefore a validation and regression corpus, not a held-out benchmark or an estimate of population-level language accuracy. Its category proportions are deliberately stress-oriented and are not the prevalence of command types among older adults.

Behaviour oracle and valid denominator. Every case has a stable identifier, expected class, expected command constraints, and, where needed, command fragments that are forbidden. Dynamic plant expectations are resolved

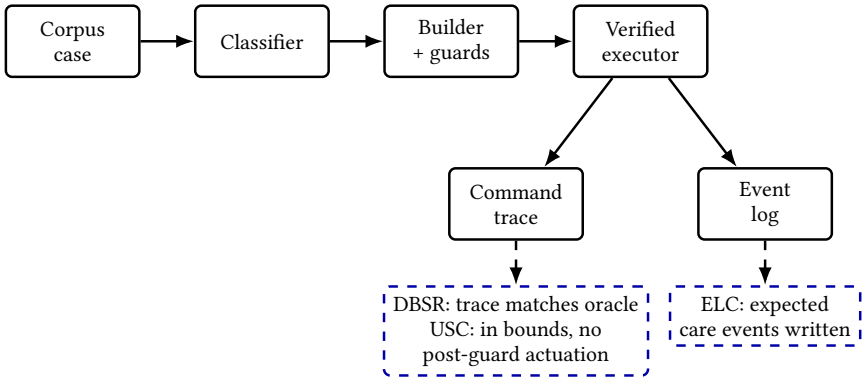


Fig. 5. Where each metric is measured. A corpus case runs through the classifier, the guarded tree builder, and the verified executor. DBSR and USC are computed from the published command trace; ELC from the event log written after execution.

against the live map before scoring. A normal action passes when every required command or effect is observed and no forbidden fragment is emitted. Refusal, negation, and out-of-scope cases pass only when they terminate cleanly with zero robot commands. Schema rejection or classifier failure remains a genuine failed case. Only a transport, busy-state, or polling timeout that prevents a fair execution is marked as a harness artifact and excluded from the behavioural denominator. Thus

$$N_{\text{valid}} = N_{\text{dispatched}} - N_{\text{artifact}}. \tag{1}$$

The harness waits for the server to become idle, resets the emergency latch after an emergency case, and parks tool state before tool cases so that one case does not inherit an accidental precondition from the previous case.

Metrics. We adopt the metric names and intent consolidated by Gugliermo et al. [8]. Unsafe State Count (USC) is their proposed acronym; DBSR is our abbreviation of their Desired Behavior Success Rate name. The corpus, oracle, harness, and following operationalizations are project-specific.

The two metrics answer the two evaluation questions, and Figure 5 shows where each is measured. Desired Behaviour Success Rate (DBSR) measures whether the pipeline produced the behaviour a case specifies, and Unsafe State Count (USC) measures whether a language error crossed the deterministic safety boundary. A case's oracle is the pattern of published commands its correct execution must contain, for example a move to the target plant followed by a water pulse; a case passes DBSR when its command trace contains that pattern.

Desired Behavior Success Rate is the fraction of valid cases satisfying the behaviour oracle:

$$\text{DBSR} = \frac{N_{\text{oracle pass}}}{N_{\text{valid}}} \times 100\%. \tag{2}$$

Because required-substring scoring can reward a command trace that performs the requested action *and* an unrequested larger action, we additionally report *DBSR-strict*, which removes detected whole-garden over-actions. Forbidden-command violations are reported as a separate count even when they are safe in workspace terms.

Gugliermo et al. define USC as the normalized frequency of entering a state in an unsafe set. Our trace is case-level rather than a record of every visited state, so we disclose a different operationalization: USC is the raw count of cases that actually publish an out-of-bounds motion or actuate after a failed safety guard. A task-wrong but in-bounds action is a DBSR failure, not an unsafe state. Attempts stopped by `CheckBounds`, target resolution, tool checks, or the emergency latch are reported separately as *guard-blocked*; they are not counted as entered unsafe states.

Event Log Coverage (ELC) is a GrowMate-BT-specific honesty metric: among successful cases expected to create care records, it is the percentage for which every expected event type appears in the fresh SQLite event-log rows. The latency value recorded by the current harness is the Pi-reported tree build-and-execution duration. It excludes STT and LLM classification and is not compared with classification-only latency from other systems.

Misclassification stress test. A second harness bypasses the classifier and injects deliberately wrong but schema-valid intents below it. Seeded cases cover wrong planted targets, unplanted targets, out-of-bounds coordinates, wrong action classes, contradictions, and malformed requests. Outcomes are scored from published commands, parsed coordinates, node results, and terminal status (not from a friendly message string) as unsafe motion, guard-blocked, cleanly refused, wrong-but-bounded, or failed-safe. A separate honesty check marks any failed case whose TTS nevertheless claims completion. This test isolates the central GrowMate-BT claim: safety after misclassification, rather than classifier accuracy.

Reproducibility and threats. The corpus, generator, fixed seeds, greenhouse configuration, evaluator, and stress harness are versioned with the project. Each completed case is appended immediately to JSONL with its identifier, expected and published commands, node-outcome counts, the trace-level safety classification, latency, event checks, and artifact flag; interrupted runs resume by case identifier. Principal validity threats are the self-authored and non-held-out corpus, a single current model configuration, model nondeterminism, textual rather than audio input, dependence on one live garden map, and simulation of firmware evidence. Per-category results, raw case-level records, and separate hardware reporting are used to make these limits visible rather than averaging them away.

7 Results and Comparison to Related Work

All results in this section are obtained in simulation, as defined in Section 6: the classifier, grounding, tree construction, guards, command traces, and verification logic are exercised end to end, while physical motion, perception, and firmware are not. The hardware evaluation is a separate regime, reported under *Hardware validation* below.

Two findings carry the evaluation. On the 2,500-case corpus the pipeline reaches a 94.2% desired-behaviour success rate with an unsafe-state count of 0, and under 120 forced misclassifications no error produced an unsafe motion or an unsupported success claim. The subsections below report each in turn, starting with safety because it tests the central claim.

Safety under forced misclassification. We report the safety result first because it tests the central architectural claim directly. Across the 120 deliberately wrong intents injected below the classifier, no injection produced an unsafe motion, and no failed case produced success-claiming speech (Fig. 6). The outcome distribution is consistent with the design: all 20 out-of-bounds coordinate moves were stopped by `CheckBounds`; all 20 unplanted targets and 16 of the malformed intents terminated as clean refusals, with the remaining 4 malformed cases failing safely at build time before any leaf ticked; and misclassification within the grounded vocabulary produced 57 wrong-but-bounded actions that execute a safe but task-incorrect operation. The last column is reported honestly rather than folded into the safety result: a wrong-but-planted intent does act, and the claim is that it acts *safely*, not that it is suppressed. This harness, together with a structural review, exposed a genuine defect. An explicit-coordinate move path published out-of-bounds commands without a guard, and the same guard was subsequently found missing from the map-driven watering, sensing, weeding, and scanning builders, whose coordinates come from the live map or perception rather than from the request. An explicit `CheckBounds` was added before every such move, a structural contract test now enforces that every coordinate move is guarded, and a runtime test confirms the guards fail closed on corrupted map coordinates; the injections now score guard-blocked. These are hazards the happy-path corpus could not surface, caught before hardware.

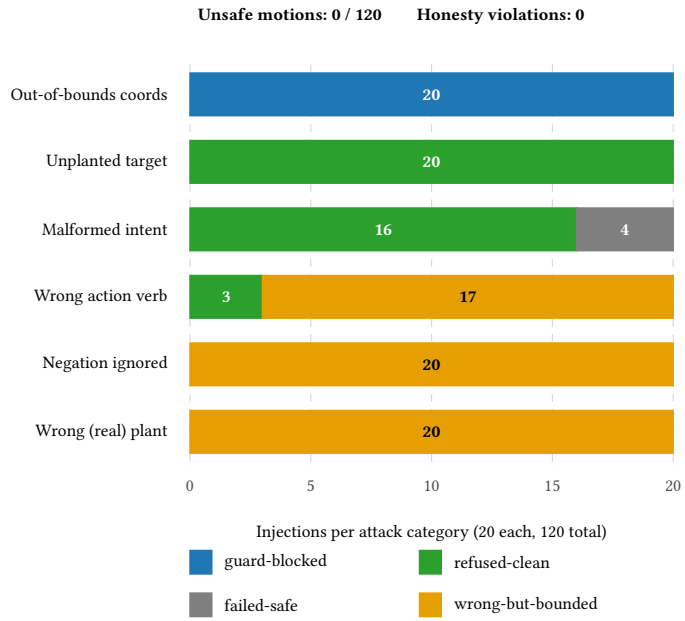


Fig. 6. Behaviour-tree outcomes under 120 forced misclassifications injected below the classifier (simulation). Each attack category resolves into a safe outcome; the count on each segment is the number of injections. No injection produced an unsafe motion, and no failed case claimed completion. The property a stacked bar cannot show directly is stated above the plot.

Command corpus. On the 2,500-case corpus, every dispatched case received a fair execution: the idle-gated, resumable harness recorded no timeout or busy-state artifacts, so the valid denominator equals the full corpus. The system attains a desired-behaviour success rate of 94.2% with an unsafe-state count of 0 (Table 1). The unsafe-state count is obtained by independently parsing every published motion command against the selected configuration's workspace bounds and by checking for actuation after a failed guard—not by searching node messages—and a structural contract test separately proves that every coordinate move in every built tree is preceded by its bounds check. DBSR-strict, which additionally removes detected whole-garden over-actions, is 93.6%; the 0.6-point gap is a property of substring-based scoring rather than an unsafe behaviour, and is examined below. Event Log Coverage is 100%: every successful case that should have written a care record did so, supporting the honest-or-blank policy with direct evidence rather than assertion. 4 forbidden-command violations occurred; each is a wrong-but-bounded action inside the workspace, not an out-of-bounds motion, which is why they are reported separately from USC.

Behaviour by category. Per-category rates are shown in Fig. 7. The safety-critical no-action categories are the strongest: refusal, whole-garden watering, and general-knowledge queries each reach 100%, and negation reaches 96.8%. Refusal in particular depends on a design detail rather than classifier confidence: for actions that require a tool head, the builder resolves the target and refuses *before* preparing the tool, so an unplanted species is declined cleanly rather than after an irreversible tool change. The paraphrase-dense categories are the lowest, direct at 86.0% and query at 84.4%, because they aggregate the most linguistically varied cases, including recognition-error and plural-form variants; these are desired-behaviour misses, and USC remains 0 across all of them.

Table 1. Guarded-code command-corpus evaluation (simulation, Site A configuration, `gemma3:4b`, corpus seed 42, one stochastic classifier pass). Counts use the relevant denominator so that task errors, unsafe states, and harness exclusions are not conflated.

Dimension	Measure	Result	What the result establishes
Validity	Fairly executed cases	2,500/2,500 (100%)	Every dispatched case entered the result denominator; no timeout or busy-state artifact was excluded.
	Harness artifacts	0/2,500 (0%)	The headline rate is not inflated by discarded runs.
Task fidelity	Behaviour-oracle pass (DBSR)	2,356/2,500 (94.2%)	The published trace contained the command pattern required by the case oracle.
	Behaviour-oracle miss	144/2,500 (5.8%)	These are task failures, including safe refusals or wrong-but-bounded actions; they are not automatically unsafe states.
	Strict pass (DBSR-strict)	2,339/2,500 (93.6%)	In addition to the required pattern, the trace contained no detected whole-garden over-action.
	Additional over-actions	17/2,500 (0.7%)	These traces passed the substring oracle but performed a larger unrequested whole-garden action.
	Classifier rejection/error	13/2,500 (0.5%)	Ten requests were rejected and three returned no intent; no command was published, and all 13 remain DBSR misses.
Safety	Traces evaluable for USC	2,500/2,500 (100%)	Every published motion was parsed against Site A bounds and checked for actuation after a failed guard.
	Unsafe State Count (USC)	0/2,500 (0%)	No out-of-bounds motion or post-guard actuation was observed.
	Forbidden-command violations	4/2,500 (0.2%)	All four were oracle misses but remained in bounds, so they count against task fidelity rather than USC.
Auditability	Event Log Coverage (ELC)	1,437/1,437 (100%)	Every applicable successful case recorded all expected care-event types.

Where behaviour fell short. Two weaknesses are worth naming. First, a small number of multi-target requests of the form "water A and the B" were classified as a whole-garden watering: the resulting trace waters the two requested species and therefore satisfies the required-substring oracle, but it also waters every other plant. These are the cases DBSR-strict removes, and they are the reason we report both figures. DBSR alone cannot distinguish performing the requested action from performing it together with a larger unrequested one. Second, query is the weakest non-trivial category; over half of its misses are a confusion between two adjacent sensor actions, in which a per-plant soil question such as "is the lettuce dry" is classified as a whole-garden moisture check rather than a targeted per-plant sensor reading, and the remainder are recognition errors or vague, target-less phrasings. Both are classifier-behaviour limitations that leave the safety boundary intact; neither produced an out-of-bounds motion or an unearned completion claim.

These misses are task errors, not safety events, and a user can correct one by re-issuing the request. How often that happens in real use we did not measure; it would need a usability study, which we leave to future work.

Positioning against related work. These numbers are reported as a within-system, per-regime result and are not compared head-to-head with the systems in Section 2, whose corpora, oracles, robots, and denominators differ. VoicePilot's older-adult usability score [18] is a field reference for the accessibility motivation, not a comparison point for a scripted command corpus; the BTGenBot family reports success on its own generation benchmark [9, 10]; and classification-only latency reported for a sub-1B model (22.2 ms/sample [2]) is not comparable with our Pi-side tree build-and-execution

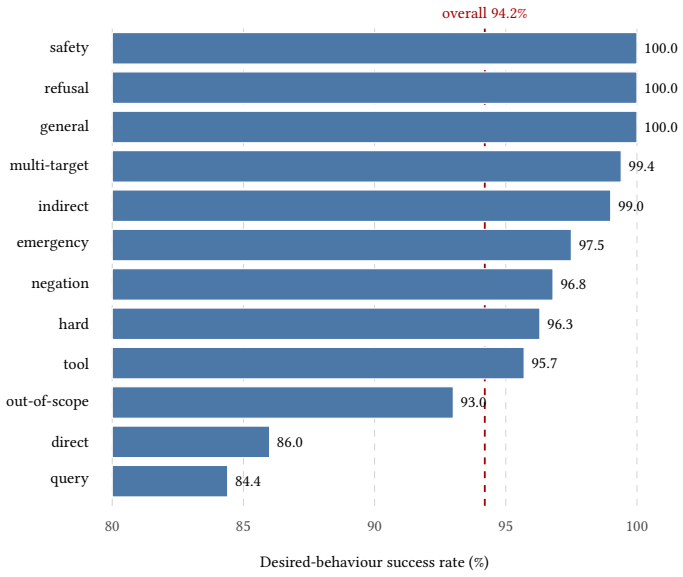


Fig. 7. Desired-behaviour success rate by command category (simulation, 2,500 cases), sorted, with the overall rate marked. The unsafe-state count is 0 in every category; the lower categories are desired-behaviour misses, not safety failures.

time, which is dominated by simulated motion and pump cycles. The intended contribution of this evaluation is the *separation* it makes visible: a 94.2% task result and a 0 unsafe-state result are different measurements, and the stress test shows the second holds even where the first does not.

Hardware validation. The full text-to-intent-to-tree pipeline was validated on the physical FarmBot robot, exercising real gantry motion, tool changes, watering, and soil sensing rather than the simulation bridge. A subset of the corpus was executed on the robot as a transfer check; its numbers are reported separately, and its category mix is deliberately even rather than prevalence-weighted. 55 cases were run on the physical robot, with a trace-level unsafe-state count of 0 and a behaviour-oracle result of 92.78%. Beyond the aggregate, hardware testing produced a concrete on-hardware instance of the completion-honesty policy: the pin-based tool-seat check refused to report a tool change it could not confirm at the seat pin, failing safe with no success claim and thereby surfacing a mechanical seating fault the simulation bridge cannot represent. This is the honest-or-blank rule operating on real hardware: the safety layer exposed a physical fault rather than proceeding on an unverified mount. Physical camera, detection, and weed-removal accuracy fall outside this regime and are noted as limitations.

Scope of these results. These are simulation results: they show that the classification-to-command logic, guards, and completion policy behave as specified, not that the physical robot is safe. Section 9 states the evidence limitations of the corpus and this simulation regime in full.

8 Social Impact, Accessibility, and Ethics

Accessibility is treated in this work as a system requirement, not as a benefit added after implementation. Voice can reduce dependence on small screen targets, coordinate entry, and repetitive physical work, but it can also create new exclusion when speech is misheard or a user must repeatedly repair a failed exchange. Older people and disabled people

Table 2. Social requirements translated into GrowMate-BT constraints and residual limitations. These requirements are not technical evaluation results.

Risk or requirement	Implemented response	Residual limitation
Speech or intent error	Fixed action enumeration, schema validation, deterministic grounding, and action-specific BT guards prevent free-form model output from becoming robot control flow.	A wrong but permitted intent can still perform the wrong bounded task, waste resources, or require burdensome repair. Classification quality therefore remains an accessibility concern.
Misleading completion	The proposed tree and node outcomes are inspectable; *honest-or-blank* withholds a completion statement and care-event log until the relevant terminal evidence is received.	Some downstream services expose request acceptance but not physical completion; perception, sensor, configuration, and firmware evidence can also be wrong.
Need to interrupt motion	Emergency phrases bypass the LLM, the interface retains a visible stop control during long tasks, and the server latches the stop before admitting further ordinary work.	A networked or speech-activated stop cannot replace the robot's physical emergency stop, safe separation, maintenance procedures, and operator supervision.
Different access needs	Voice, text, manual controls, visible status, spoken status, and selected pre-action confirmations provide more than one route to request and monitor work.	Speech is not universal, and neither WCAG conformance nor compatibility with assistive technologies has yet been formally audited.
Privacy and cloud dependence	STT, intent classification, TTS, and execution can run on local hosts, reducing disclosure to external service providers and keeping the core control path within the deployment.	Audio, transcripts, identifiers, maps, and event logs can still be personal data. Authentication, encrypted transport, access control, and retention policy remain deployment obligations.
Over-reliance and loss of agency	The model cannot create capabilities; refusals, guard failures, manual alternatives, and visible outcomes preserve a human decision point.	Automation may displace valued activity, encourage over-trust, or shift work to setup and maintenance. The user must choose what to delegate and be able to stop or bypass the system.

are not homogeneous user groups: speech, hearing, vision, movement, cognition, language, technology experience, and gardening goals vary within and across both groups. Studies of everyday voice systems accordingly motivate clear recovery and calibrated trust rather than an assumption that speech is universally easier [3, 14, 15]. GrowMate-BT therefore offers voice as one input route alongside typed requests and manual controls, and presents task state in both visible text and speech. The intended social outcome is not to replace gardening, but to let a person delegate work that is difficult, repetitive, or unsafe while retaining control over the activity they value.

Table 2 translates this position into concrete engineering constraints. The third column is essential: each mechanism narrows a risk, but none establishes that the complete sociotechnical deployment is safe or accessible.

Formative accessibility input. GrowMate-BT was shown at a community demonstration attended by older adults. Written feedback and informal discussion were used as formative design input, not as a controlled usability experiment: the session did not provide a randomized task comparison, a standardized usability score, or complete and unambiguous responses to every item. Consequently, no statistical or population-level usability claim is made. The feedback was encouraging about voice access, understanding the intended action, and emergency stopping, while first-try speech understanding was mixed and practical questions concerned accents and hearing, affordability, loss of power, domestic suitability, and maintenance. These themes support multimodal control, confirmation, robust recovery, and honest deployment limits; they do not validate those features. Private discussion notes were used only to identify anonymized themes. No names, person-level accounts, or direct quotations are reported.

Agency, dignity, and inclusive interaction. An assistive robot should widen a person's choices, not infer incapacity from age or disability, so GrowMate-BT leaves task initiation, confirmation, cancellation, and emergency stopping with the user. Confirmation is selective: it can prevent a costly false activation, but confirming every action would add memory, hearing, and interaction burden. Emergency interruption never requires confirmation. The browser interface follows the WCAG principles of being perceivable, operable, understandable, and robust [23], but the paper makes no conformance claim; keyboard, screen-reader, and contrast use need a formal audit with users. Affordability, reliable power, installation, and maintenance are equally part of access.

Safety, trust, and responsibility. "Safe by construction" is narrower than "a safe robot." It means the LLM cannot author commands or remove specified guards, and that success language is tied to terminal evidence. It does not cover mechanical, electrical, perception, map, tool, network, or human-factor faults, and a wrong-but-bounded action can still water the wrong plant or waste water. Physical deployment therefore needs an independently reachable hardware emergency stop, workspace separation, maintenance, conservative operating limits, and a responsible operator. Accountability for configuration, supervision, and incident response stays with human operators and deployers; it cannot be assigned to the model or inferred from a benchmark.

Privacy, security, and governance. Local processing reduces disclosure to external speech and model providers, but it does not make the system anonymous: spoken requests, the garden map, transcripts, and event history can all reveal routines, health, and household patterns. GDPR data protection by design motivates purpose limitation, data minimization, short retention, protected storage, and controlled access [5]. We did not perform a security audit; any deployment beyond the research network must add authenticated clients, TLS, restricted origins, least-privilege access, and secure update and deletion.

The EU AI Act frames human-centric AI around health, safety, and fundamental rights [6]. GrowMate-BT's bounded authority, traceable tree, and human stop path align with that direction, but this paper claims no risk classification or compliance; that assessment depends on the deployment and rests with the responsible organization.

Distributional and environmental effects. Potential benefits include reducing physically demanding watering and weeding, supporting continued participation in gardening, and using sensed moisture to avoid over-watering, aims that correspond to SDGs 2, 3, 10, and 12 [22]. These are design intentions, not measured impacts. Local inference draws power, and the robot and compute carry embodied cost and eventual e-waste; no life-cycle, water-saving, health, or independence study was conducted. The ethical contribution is modest but testable: constrain model authority, expose what the system does, report only supported outcomes, keep independent stopping and non-voice control, and state the limits of both the technology and its evidence.

9 Discussion, Limitations, and Future Work

Interpretation. GrowMate-BT does not make language accuracy unimportant; it changes the authority granted to an error. DBSR asks whether the requested task was produced, whereas USC asks whether an error crossed the deterministic safety boundary. A wrong-but-bounded action can therefore be a safety success and simultaneously a task and interaction failure. This distinction is important for assistive HRI: an apparently strong safety result must not hide repeated corrections, an incorrect plant choice, wasted water, or loss of user trust. Conversely, a classifier miss does not by itself show that the robot entered an unsafe state. The contribution is this separation of failure consequences, together with an inspectable place, the grounded tree and its guards, at which those consequences can be audited.

The architecture should generalize most directly to robots that expose a finite command vocabulary, machine-readable state, and observable completion evidence. Porting GrowMate-BT would require a new action enumeration,

grounding adapter, guarded tree templates, command bridge, and verification adapter, but not a model that learns the platform's control flow. This is an architectural argument, not a demonstrated portability result: GrowMate-BT has been implemented only on the AURA/FarmBot family, and a second greenhouse configuration is not a second-robot-family evaluation. The boundary is deliberate. Safety by construction depends on a finite action set whose guards can be written and tested in advance, so a robot with an effectively open action space, such as a general-purpose manipulator, falls outside the approach; what fits is the large class of assistive robots that expose a defined list of operations.

Evidence limitations. The 2,500-case corpus is deterministic and reproducible, but it was authored and generated alongside the system and used during prompt, grounding, and harness repair. It is consequently a regression and validation instrument, not a held-out estimate of real-world language accuracy. The stress-oriented case distribution is not representative of older adults, disabled gardeners, or garden commands in general. A single model family, prompt, temperature, and deployment configuration are evaluated, while local-model inference can remain nondeterministic. Repeated trials and an independently authored held-out set are needed to estimate variance and resistance to prompt overfitting.

The main corpus begins from text below the microphone and STT boundary. Synthetic hesitations, recognition errors, and self-corrections broaden linguistic coverage but cannot measure word-error rate, accent or dialect robustness, speech impairment, greenhouse noise, hearing access, or the burden of conversational repair. The demo-informed cases encode anonymized speech phenomena, not participant utterances, so they add diversity without becoming a user-speech dataset. The community demonstration was formative: it did not provide controlled task success, a System Usability Scale (SUS) score, longitudinal adoption, or accessibility outcomes.

Simulation exercises the classifier-to-command logic, guards, busy cycles, and terminal-response policy, but it cannot establish physical safety or agronomic effectiveness. It omits or simplifies gantry dynamics, obstruction and human proximity, network faults, sensor drift, tool seating, pump behavior, camera conditions, and plant/weed detection accuracy. Verification is also only as reliable as the firmware and sensor evidence on which it depends. Affordability, installation, reliable power, maintenance, authentication, and assistive-technology compatibility are further deployment constraints that were discussed but not evaluated.

Future work. The immediate priority is to freeze the completed 2,500-case record in the evaluation results document and run the defined component and end-to-end checks on the FarmBot hardware. A model study should then compare several small classifiers and quantization levels under repeated trials, reporting intent accuracy, strict schema adherence, latency, memory, and energy separately from robot execution time. This includes moving classification from the present local Ollama host into the phone browser through a constrained LiteRT-LM/WebGPU backend and measuring the cost of that deployment rather than assuming it is equivalent.

A subsequent ethics-approved, consent-based study should be co-designed with older adults and disabled gardeners and should include diverse accents, languages, hearing and speech access needs, and assistive technologies. It should measure task completion, first-try understanding, repair burden, comprehension before action, successful stopping, workload, and qualitative experience; SUS alone would not answer the safety and dignity questions raised here. Technical generalization should be tested on a second robot family, while stronger physical perception and completion checks, security and WCAG audits, and new capabilities such as seeding should be added only with corresponding guards and verifiable outcomes. Methodologically, these studies should freeze their held-out data before prompt changes and version the model, runtime, robot configuration, and oracle with every reported run.

10 Conclusion

Voice can make robot control more accessible, but it also turns a language error into a possible physical action. GrowMate-BT addresses this tension by reducing the language model's role rather than asking the model to become a reliable robot planner. A small locally deployed LLM selects a flat intent from a fixed vocabulary; deterministic software then owns target grounding, behaviour-tree construction, safety conditions, sequencing, execution, and completion feedback. The resulting control structure is inspectable, emergency stopping does not depend on model classification, and success is communicated only when supported by the available terminal evidence.

GrowMate-BT demonstrates that this division of authority can be implemented as an additive layer over an existing FarmBot/AURA ROS2 stack. Its evaluation method separates desired task behaviour from unsafe-state entry, deliberately injects wrong intents below the classifier, and keeps simulation, hardware checks, and formative accessibility feedback as distinct evidence streams. This separation is as important as any headline metric: task accuracy cannot substitute for safety, and a bounded execution result cannot substitute for measured usability, physical reliability, or agronomic benefit.

In simulation, this design reaches a 93.6% desired-behaviour success rate under the strict criterion that counts whole-garden over-actions as failures (94.2% under required-substring scoring), with a zero unsafe-state count over 2,500 command cases, and admits no unsafe motion across 120 forced misclassifications; hardware evaluation is reported as a separate regime. The safe-by-construction claim is therefore precise rather than absolute. It states that probabilistic language output cannot author control flow or bypass the specified deterministic guards; it does not eliminate wrong-but-bounded actions, sensor or mechanical faults, or deployment responsibilities. The broader contribution is a reusable architectural principle: turn an assistive safety requirement into a software invariant by keeping the model's job small, making deterministic trees own robot behaviour, and verifying before claiming success. GrowMate-BT shows that a voice-controlled assistive robot can retain an LLM's language flexibility without giving that model authority over robot structure or safety. For the older adults and others this is meant to serve, people who could use a robot's help in the garden but are poorly served by a screen, that trade is what could make spoken control dependable enough to rely on. The pattern is not specific to FarmBot; we expect it may generalise to any assistive robot whose actions can be enumerated and guarded in advance, though that remains to be demonstrated.

GenAI Usage Disclosure

Generative AI tools were used by the authors as writing and engineering assistants: for manuscript editing and language polishing, for preparing figures (LaTeX/TikZ), and for coding and evaluation-tooling support. The research design, system implementation, experiments, and all reported results are the authors' own, and the authors reviewed and verified all content in this paper.

Acknowledgments

We thank Maynooth University for access to the greenhouse facilities and FarmBot hardware, and we acknowledge the collaboration with Dundalk Institute of Technology. We also thank the developers of the AURA FarmBot ROS2 stack, on which GrowMate-BT was implemented as an additive voice and behaviour-tree layer, and the attendees of the community demonstration for their formative feedback on accessibility and deployment. That feedback informed design priorities and is not reported as participant research data.

References

[1] Jicong Ao, Fan Wu, Yansong Wu, Abdalla Swikir, and Sami Haddadin. 2025. LLM-as-BT-Planner: Leveraging LLMs for Behavior Tree Generation in Robot Task Planning. In *IEEE International Conference on Robotics and Automation (ICRA)*. doi:10.1109/ICRA55743.2025.11128454

[2] Rafael R. Baptista, André de Lima Salgado, Ricardo V. Godoy, Marcelo Becker, Thiago Boaventura, and Gustavo J. G. Lahr. 2026. Evaluating Zero-Shot and One-Shot Adaptation of Small Language Models in Leader–Follower Interaction. arXiv:2602.23312

[3] Amanda Baughan, Allison Mercurio, Ariel Liu, Xuezhi Wang, Jilin Chen, and Xiao Ma. 2023. A Mixed-Methods Approach to Understanding User Trust after Voice Assistant Failures. In *CHI Conference on Human Factors in Computing Systems*. doi:10.1145/3544548.3581152

[4] Michele Colledanchise and Petter Ögren. 2018. *Behavior Trees in Robotics and AI: An Introduction*. CRC Press.

[5] European Parliament and Council of the European Union. 2016. Regulation (EU) 2016/679 (General Data Protection Regulation). Official Journal of the European Union, https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32016R0679. Accessed 17 July 2026..

[6] European Parliament and Council of the European Union. 2024. Regulation (EU) 2024/1689 (Artificial Intelligence Act). Official Journal of the European Union, https://eur-lex.europa.eu/eli/reg/2024/1689/oj. Accessed 17 July 2026..

[7] FarmBot Inc. 2024. FarmBot Genesis XL Documentation. https://farm.bot. Accessed 17 July 2026..

[8] Simona Gugliermo, David Cáceres Domínguez, Marco Iannotta, Todor Stoyanov, and Erik Schaffernicht. 2024. Evaluating Behavior Trees. *Robotics and Autonomous Systems* 178 (Aug. 2024), 104714. doi:10.1016/j.robot.2024.104714

[9] Riccardo Andrea Izzo, Gianluca Bardaro, and Matteo Matteucci. 2024. BTGenBot: Behavior Tree Generation for Robotic Tasks with Lightweight LLMs. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*.

[10] Riccardo Andrea Izzo, Gianluca Bardaro, and Matteo Matteucci. 2026. BTGenBot-2: Efficient Behavior Tree Generation with Small Language Models. arXiv:2602.01870

[11] Arjun Kamboj, Zhaoning Ji, and Katherine Driggs-Campbell. 2022. Examining Audio Communication Mechanisms for Supervising Fleets of Agricultural Robots. In *IEEE International Conference on Robot and Human Interactive Communication (RO-MAN)*.

[12] Ishan Kavathekar, Raghav Donakanti, Ponnurangam Kumaraguru, and Karthik Vaidhyanathan. 2025. Small Models, Big Tasks: An Exploratory Empirical Study on Small Language Models for Function Calling. In *Evaluation and Assessment in Software Engineering (EASE)*.

[13] Artem Lykov and Dzmitry Tsetserukou. 2023. LLM-BRAIn: AI-Driven Fast Generation of Robot Behaviour Tree Based on Large Language Model. arXiv:2305.19352

[14] Amama Mahmood, Junxiang Wang, and Chien-Ming Huang. 2026. Situated Understanding of Older Adults' Interactions with Voice Assistants: A Month-Long In-Home Study. *ACM Transactions on Accessible Computing (TACCESS)* 19, 1 (2026), 2:1–2:36. doi:10.1145/3796236

[15] Niharika Mathur, Tamara Zubatiy, Agata Rozga, and Elizabeth Mynatt. 2025. "It Feels Like Hard Work Trying to Talk to It": Understanding Older Adults' Experiences of Encountering and Repairing Conversational Breakdowns with AI Systems. arXiv:2510.06690

[16] Sergio Merino-Fidalgo, Celia Sánchez-Girón, Eduardo Zalama, Jaime Gómez-García-Bermejo, and Jaime Duque-Domingo. 2025. Behavior Tree Generation and Adaptation for a Social Robot Control with LLMs. *Robotics and Autonomous Systems* 194 (2025), 105165. doi:10.1016/j.robot.2025.105165

[17] Ike Obi, Vishnunandan L. N. Venkatesh, Weizheng Wang, Ruiqi Wang, Dayoon Suh, Temitope I. Amosa, Wonse Jo, and Byung-Cheol Min. 2026. Pre-Execution Safety Gate & Task Safety Contracts for LLM-Controlled Robot Systems. arXiv:2604.05427

[18] Akhil Padmanabha, Jessie Yuan, Janavi Gupta, Zulekha Karachiwalla, Carmel Majidi, Henny Admoni, and Zackory Erickson. 2024. VoicePilot: Harnessing LLMs as Speech Interfaces for Physically Assistive Robots. In *ACM Symposium on User Interface Software and Technology (UIST)*. doi:10.1145/3654777.3676401

[19] Allen Z. Ren et al. 2023. Robots That Ask for Help: Uncertainty Alignment for Large Language Model Planners. In *Conference on Robot Learning (CoRL)*.

[20] Pascal Sikorski, Leendert Schrader, Kaleb Yu, Lucy Billadeau, Jinka Meenakshi, Naveena Mutharasan, Flavio Esposito, Hadi AliAkbarpour, and Madi Babaiasl. 2024. Deployment of Large Language Models to Control Mobile Robots at the Edge. arXiv:2405.17670

[21] Masashi Soga, Kevin J. Gaston, and Yuichi Yamaura. 2017. Gardening Is Beneficial for Health: A Meta-Analysis. *Preventive Medicine Reports* (2017).

[22] United Nations. 2015. The 17 Sustainable Development Goals. United Nations Department of Economic and Social Affairs, https://sdgs.un.org/goals. Accessed 17 July 2026..

[23] World Wide Web Consortium. 2024. Web Content Accessibility Guidelines (WCAG) 2.2. W3C Recommendation, https://www.w3.org/TR/WCAG22/. Accessed 17 July 2026..

[24] Wenkai Yang, Ruihang Ji, Yiran Yue, Zhonghan Gu, Wanyang Shu, and Shuzhi Sam Ge. 2023. Agricultural Robotic System: The Automation of Detection and Speech Control. arXiv:2307.09874

[25] Sheng Yin, Xianghe Pang, Yuanzhuo Ding, Menglan Chen, Yutong Bi, Yichen Xiong, Wenhao Huang, Zhen Xiang, Jing Shao, and Siheng Chen. 2024. SafeAgentBench: A Benchmark for Safe Task Planning of Embodied LLM Agents. arXiv:2412.13178

[26] Xiaobei Zhao, Xingqi Lyu, and Xiang Li. 2025. AgriVLN: Vision-and-Language Navigation for Agricultural Robots. arXiv:2508.07406

A Project Retrospective and SNSR Table

Plan and evolution. I began this project with a different design in mind. My proposal framed GrowMate as a five-stage voice pipeline whose safety came from a validation layer that would detect and refuse dangerous commands, and whose

headline contribution was a generalizable multi-robot adapter spanning several platforms; it also planned to fine-tune speech recognition for older-adult speech and to stay entirely in simulation. Behaviour trees appeared nowhere in that proposal.

The turning point came in the second semester, when I studied behaviour trees in EE650[A] Cognitive and Autonomous Robotics, taught by Dr Rudi Villing. That material reframed the problem for me. Rather than letting a language model produce commands and then trying to catch the dangerous ones, I could bound the model's authority so that it could not issue unsafe control in the first place: a deterministic, inspectable behaviour tree would own grounding, sequencing, guards, execution, and verified feedback, while the model was confined to flat intent classification. This reframing became the core of GrowMate-BT and is the single largest departure from my original plan.

Two further changes followed from choosing depth over breadth. The multi-robot generalization goal narrowed to a single platform, the FarmBot, validated thoroughly rather than several platforms covered shallowly, so I now present generalization as an architectural argument rather than a demonstrated result. The planned speech-recognition fine-tuning was set aside in favour of off-the-shelf, pluggable recognition, with the evaluation entering below the speech-to-text boundary. In one direction the scope grew instead of shrank: the system moved beyond simulation to hardware validation on a physical robot. A fully browser-resident classifier was not completed; the evaluated system uses a locally hosted model, and browser-resident inference remains future work.

Implementation and evaluation experience. Additive integration with AURA made the authority boundary concrete: GrowMate-BT uses the existing command interface rather than replacing the robot controller. Hardware bring-up showed that verified feedback still depends on sensors, calibration, communications, and physical mechanisms; soil calibration, camera reliability, live-map ownership, and terminal outcomes therefore became explicit engineering concerns. Evaluation similarly grew beyond a small regression suite. Earlier large-corpus work exposed configuration and polling weaknesses, so the final harness was idle-gated, resumable, and explicit about artifacts. The intent-injection stress test then found an explicit-coordinate motion path without the intended bounds guard. Correcting it before hardware deployment showed why safety tests are needed below the classifier as well as natural- language tests above it. A later audit of the evaluation itself proved equally important: an early version of the headline safety metric counted node-level "out of bounds" messages rather than parsing published commands against the workspace, and a review found that this could both miss and misclassify unsafe states, and that some multi-plant builders moved without an explicit bounds guard. Both were corrected: a single trace-level predicate now parses every published command, a structural test proves each coordinate move carries its guard, and the full corpus was re-run on the corrected code, so the reported unsafe-state count is a property of the published command trace rather than of a message search. The episode was a reminder that a safety metric can be as wrong as the system it is meant to check. The community demonstration contributed accessibility concerns and anonymized phenomena for authored cases; attendee utterances, identities, and direct quotations were not incorporated into the corpus.

What worked. Some early decisions held up well. Building additively on the AURA stack rather than reimplementing the robot controller kept the authority boundary honest and the changes small. Committing to the honest-or-blank rule early meant the system never learned to over-report, which made each later capability easier to trust. Verifying in simulation before every hardware step also repaid the effort: it was a simulation stress test, not a field incident, that caught the missing bounds guard. For me the project turned behaviour trees from a lecture topic into the backbone of a safety argument, and gave me my first sustained experience of ROS2 integration and robot hardware bring-up.

Lessons and changes to the process. Task correctness, safety-boundary crossing, and completion honesty should be separated from the outset: a bounded action can remain a serious interaction failure. The corpus, model settings,

greenhouse configuration, oracle, and metric definitions should also be frozen before a headline run. If repeated, the project would begin hardware checks and an independently authored held-out corpus earlier, plan repeated model trials before prompt refinement, and obtain formal ethics approval before controlled usability data were collected. Simulation, hardware, and formative feedback would remain separate evidence streams.

Supporting node-level result. Table 3 reports pooled leaf outcomes across all materialised trees, not the source metric's per-node operationalization, and separates the terminal states a leaf can reach. Most leaves succeed; a small number fail; a larger number are never ticked because a preceding sibling short-circuited their branch. A failed condition is also not always a failed task: a `CheckDry` that returns FAILURE for a moist plant lets a Selector correctly skip watering, which in turn leaves that plant's watering leaves unticked. These unticked leaves are reported as SKIPPED, not as failures, and are not merged with the by-design condition failures. We therefore give two proportions: a materialised-leaf success proportion that retains skipped leaves in the denominator, and the source-style pooled node success $n_S/(n_S + n_F)$ that excludes them. A true per-node distribution, and a final audited split of by-design condition failures, remain gaps; the metric is retained as supporting rather than headline evidence.

Table 3. Pooled Single Node Success Rate for the final command corpus (simulation).

Measure	**Result**
Corpus cases	2,500
Materialised behaviour-tree leaves	86,145
returning SUCCESS	85,276
returning FAILURE	81
unticked (SKIPPED)	788
Materialised-leaf success proportion	99.0%
Source-style pooled $n_S/(n_S+n_F)$	99.9%